

Question 1

Compound Assignment Operators, Type Casting, String Conversion, and Object References

Theory Component

Investigate the Java Language Specification (JLS) regarding **compound assignment operators** (e.g., `+=`, `-=`) and their interaction with **type casting**. Specifically:

1. Explain the statement: *A compound assignment expression of the form `E1 op= E2` is equivalent to `E1 = (T) ((E1) op (E2))`, where `T` is the type of `E1`*. How does this implicit cast enable operations that would otherwise require an explicit narrowing conversion? Provide a concrete primitive-type example.
2. When the right-hand operand of a `+` operator is a **reference type**, Java performs *string conversion*. Research how this conversion is defined, including the handling of `null` references. How does this interact with the `+=` operator when a `String` and an object reference are used together?
3. Discuss the rules for **casting reference types**: upcasting (implicit) vs. downcasting (explicit). What runtime checks are performed, and what exception is thrown if a downcast is invalid?

Code Component

Given the following partial definitions, write a complete Java program that answers the tasks below. Your code must demonstrate the concepts researched.

```
interface Printable {
    void print();
}

class Document implements Printable {
    private String title;
    private double version;
    private Object metadata;    // reference field, not initialized

    public Document(String title, double version) {
        this.title = title;
        this.version = version;
        // metadata intentionally left unassigned
    }
}
```

```

@Override
public String toString() {
    return title + " v" + version;
}

@Override
public void print() {
    System.out.println("Printing: " + this);
}
}

```

1. **Object creation and shorthand with strings:** In `main`, create a `Document` object `doc`. Use the `+=` shorthand to build a `String` description that concatenates a literal, `doc`, and `doc.metadata`. Print the result before and after assigning `metadata`. Explain the output, especially the representation of `null`.
2. **Shorthand with numeric field and casting:** Attempt to increment `version` by an `int` value using `doc.version += 1;`. Will this compile? Research why or why not (hint: accessibility). Modify the class to encapsulate `version` properly (using a setter), then use a local variable and a compound assignment to increase the version by a `double` value. Show the implicit cast behaviour when the right operand is an `int`.
3. **Reference casting and member access:** Assign `doc` to a `Printable` reference. Use this reference to call `print()`. Next, cast the `Printable` reference back to `Document` and call `toString()`. Then, deliberately attempt an invalid downcast (e.g., cast a `Printable` that is not a `Document` to `Document`) and report the outcome. Explain why the compiler does not prevent this but the JVM enforces type safety.
4. **Default values:** Access the `metadata` field before it has been explicitly assigned. What is its value? Relate this to the JLS rules for **default values of reference fields** and verify with code.